NLP • DAY 2

N-Grams, POS Tagging & Lexical Resources

From word sequences and probability to grammatical structure and meaning

Dr. Minni Jain | Department of Computer Science & Engineering | Delhi Technological University

Today's Agenda

1

N-Grams & Probability

Building blocks, MLE, and how models estimate likelihood

2

Applications & Challenges

Language modeling, spelling correction, data sparsity, smoothing

3

Part-of-Speech Tagging

Rule-based, statistical (HMM), and neural approaches

4

Decoding with Viterbi

Finding the most likely tag sequence efficiently

5

WordNet

A graph-structured lexical database of meaning and relations

6

Centrality & Word Sense Disambiguation

Using graph structure to resolve ambiguous meaning

N-Grams: The Building Blocks

A contiguous sequence of n items (words, in our case) from a text

```
Example sentence:  "Data drives smart decisions"   →   tokens: [Data, drives, smart, decisions]
```

Unigram (n = 1)

Single tokens

```
Data | drives | smart | decisions
```

Bigram (n = 2)

Overlapping pairs of tokens

```
Data drives | drives smart | smart decisions
```

Trigram (n = 3)

Overlapping triplets of tokens

```
Data drives smart | drives smart decisions
```

Estimating N-Gram Probability

Maximum Likelihood Estimation: count how often things happen together

General idea (bigram case):

$$P(word_n \mid word_{n-1}) = count(word_{n-1}, word_n) \div count(word_{n-1})$$

The probability of a word given the previous word is simply how often that pair occurred, divided by how often the first word occurred alone.

Worked Example

Suppose in a training corpus:

- the phrase "data drives" appears 6 times
- the word "data" appears 10 times (on its own, as the first word of any bigram)

$$P(\text{"drives"} \mid \text{"data"}) = 6 / 10 = 0.6$$

Applications of N-Grams

A simple idea with a surprisingly wide reach

1

Text Representation

Represent documents as frequency vectors of n-grams (“bag-of-n-grams”). E.g., bigrams like “free money” or “act now” are strong spam signals.

2

Language Modeling

Predict the next word from the previous n−1 words. Powers autocomplete, predictive text, and speech recognition.

3

Spelling Correction

Character-level n-grams catch misspellings — a typo usually still shares most of its character n-grams with the correct word.

Spelling Correction with N-Grams

Combine candidate generation with language-model scoring

1. Find dictionary words within a small edit distance of the misspelled word — these are candidates
2. For each candidate, substitute it into the sentence and score the resulting phrase with a bigram/trigram language model
3. Pick the candidate that gives the sentence the highest overall probability

Worked Example

```
Typed sentence:  "I want peace of cake"
```

Candidates for "peace": peace, piece (both within 1 edit of the typo pattern)

Bigram model score: "piece of cake" → high probability "peace of cake" → low probability

Result: "piece" is selected — correcting to "I want piece of cake"

Edit Distance (Levenshtein Distance)

The minimum number of edits needed to turn one string into another

Three allowed operations:

- Insertion — add a character
- Deletion — remove a character
- Substitution — swap one character for another

Example

`"helo"` → `"hello"` `distance = 1`

one insertion

`"recieve"` → `"receive"` `distance = 2`

two substitutions (swap positions of i/e)

Challenges: Sparsity & Corpus Dependency

N-gram models are only as good as the data they've seen

Data Sparsity

As n increases, the number of possible n-grams grows exponentially — most of them will never appear in any training corpus, however large.

Consequence: the model assigns zero probability to any sequence it hasn't seen — even a perfectly reasonable one.

Corpus Dependency

The model's quality is tied entirely to the corpus it was trained on — a model trained on news text will handle casual chat poorly.

Consequence: many real, valid word sequences from a new domain remain unseen, leading to poor or unstable predictions.

Smoothing Techniques

Redistributing probability mass so unseen sequences aren't impossible

Laplace (Additive) Smoothing

Add 1 to every count before computing probabilities, so no sequence ever gets exactly zero.

Add-k Smoothing

A gentler version of Laplace — add a small constant k (e.g., 0.1) instead of 1, to avoid overcorrecting.

Back-off (e.g., Katz)

If a higher-order n-gram has zero count, fall back to a lower-order one (trigram → bigram → unigram).

Interpolation

Blend probabilities from multiple n-gram orders together, weighted by how reliable each order is.

Part-of-Speech (POS) Tagging

Assigning each word its grammatical role, based on meaning, position, and context

Example: "The quick brown fox jumps over the lazy dog"

The	DT	Determiner	quick	JJ	Adjective	brown	JJ	Adjective
fox	NN	Noun (singular)	jumps	VBZ	Verb, 3rd person singular	over	IN	Preposition
the	DT	Determiner	lazy	JJ	Adjective	dog	NN	Noun (singular)

Methods for POS Tagging

Three generations of approaches

1

Rule-Based

Handcrafted grammar rules applied directly to words and their neighbors.

```
If a word ends in "-ing" and
follows "is" → tag as VBG
```

2

Statistical

Probabilities learned from large tagged corpora.

```
HMM · Maximum Entropy ·
Conditional Random Fields
```

3

Neural

Word embeddings combined with deep learning architectures.

```
BiLSTM taggers ·
Transformer-based (e.g., BERT)
taggers
```

Rule-Based POS Tagging in Practice

Tags are assigned by applying manually written linguistic rules

R1 A word ending in "-ing", not preceded by "is"/"was" → likely VBG (present participle)

R2 A word following a determiner (DT) that is not itself an adjective → likely a noun (NN)

R3 A word ending in "-ly" → likely an adverb (RB)

R4 A word ending in "-ed", following a pronoun → likely a past-tense verb (VBD)

Strength: transparent and interpretable. Weakness: brittle — breaks on exceptions and doesn't scale to new domains without rewriting rules.

Statistical POS Tagging & the HMM

Learning tag probabilities from annotated corpora

Statistical tagging replaces handwritten rules with probabilities learned from a large corpus of text that humans have already tagged — the model learns which tags are likely, given the word itself and the tags around it.

A Hidden Markov Model (HMM) is the classic tool for this: it treats the sequence of true tags as “hidden” states that generate the words we actually observe.

States

The possible tags (NN, VB, DT, JJ, …) the model can be in

Transition Probabilities

How likely one tag is to follow another (e.g., DT → NN is common)

Emission Probabilities

How likely a given tag is to produce a specific word

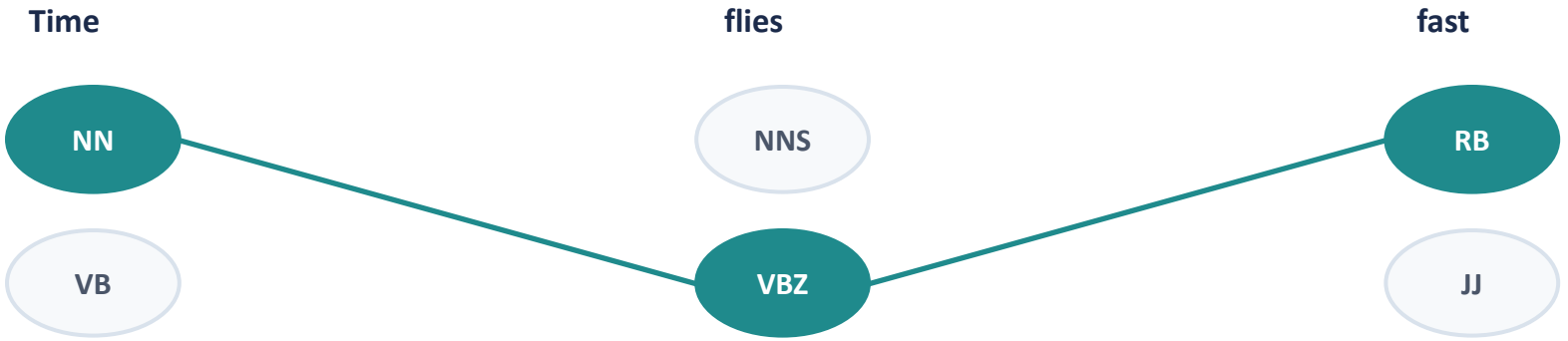
Initial Probabilities

How likely each tag is to start a sentence

Decoding with the Viterbi Algorithm

Finding the single most likely tag sequence, efficiently

Checking every possible tag sequence for a sentence would be exponentially expensive. Viterbi is a dynamic-programming algorithm that finds the best path through the space of tag sequences without ever enumerating them all — by keeping only the best partial path to each state at each step.



Highlighted path shows the highest-probability tag sequence Viterbi selects: Time/NN → flies/VBZ → fast/RB

Viterbi for Spelling Correction

The same decoding idea, applied to choosing the best sentence-wide correction

1. Take the sentence containing a suspected error as input
2. Generate candidate words (via edit distance) to serve as possible “states” at that position
3. Compute transition probabilities (language model) and observation probabilities (typo likelihood) for each candidate
4. Compute the Viterbi score at each step, keeping only the best path so far
5. Trace back through the chosen path to recover the corrected sentence

WordNet: A Lexical Database

Organizing English vocabulary by meaning, not just spelling

What It Is

A large lexical database that groups English words into sets of synonyms (“synsets”), each representing one distinct concept, and links those sets through meaning-based relations.

Synset example:
{ car, automobile }

Both words point to the same underlying concept, even though they're different words.

Core Relation Types

Hypernymy (is-a)

`beverage → drink`

Hyponymy (kind-of)

`tulip → flower`

Meronymy (part-of)

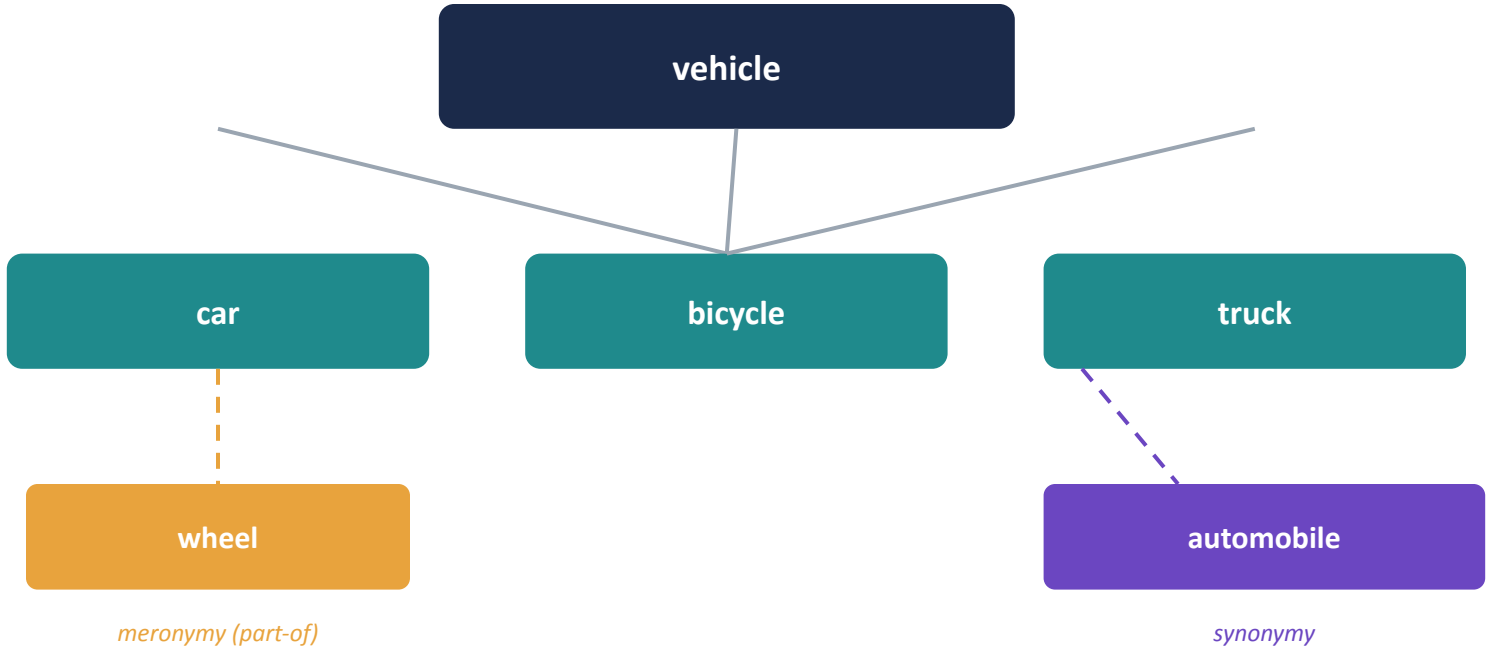
`engine → automobile`

Antonymy (opposite)

`fast ↔ slow`

WordNet as a Graph

Nodes are concepts; edges are the relationships between them



- Hypernym/hyponym links form a directed hierarchy (broader → narrower concepts)
- Synonymy and antonymy links are symmetric (undirected)
- Multiple relation types can connect the very same pair of concepts

What WordNet's Structure Enables

A graph of meaning becomes a tool for several core NLP tasks

1 **Word Sense Disambiguation**

Deciding which meaning of an ambiguous word applies in context

2 **Semantic Similarity**

Measuring how close two words or concepts are in meaning

3 **Ontology Building**

Constructing structured knowledge hierarchies from concept relations

4 **Knowledge Graph Integration**

Linking lexical meaning into larger structured knowledge bases

Using Graph Centrality for Word Sense Disambiguation

The most “connected” sense in context is often the correct one

1. Treat WordNet as a graph: nodes are word senses (synsets), edges are semantic relations
2. For an ambiguous word in a sentence, build a subgraph of all candidate senses for every word in that context
3. Compute a centrality score for each candidate sense — how well-connected it is to the other words' senses
4. Select the sense with the highest centrality as the most contextually appropriate meaning

This graph-connectivity approach to unsupervised sense disambiguation has been studied extensively in the NLP research literature.

Centrality Beyond Word Sense Disambiguation

The same idea — "important nodes are well-connected" — shows up across NLP

Keyword Extraction

Words with high centrality in a co-occurrence graph often represent the main concepts of a document

Text Summarization

Sentences that are most central in a sentence-similarity graph are selected as summary candidates

Information Retrieval

A term's centrality in a semantic network can improve how results are ranked

Word Sense Disambiguation

As seen above — the most central candidate sense is typically the intended one

Key Takeaways

- N-grams turn text into countable sequences, powering language modeling, spam detection, and spelling correction
- Smoothing prevents unseen sequences from being treated as impossible — essential given data sparsity
- POS tagging assigns grammatical roles via rule-based, statistical (HMM), or neural methods
- Viterbi decoding finds the best tag sequence efficiently, without checking every possibility
- WordNet organizes vocabulary as a graph of meaning — synsets connected by relations like hypernymy and meronymy
- Graph centrality turns that structure into a tool for disambiguation, keyword extraction, and summarization